06 ottobre 2026

Dal problema aziendale a ML, LLM e dashboard

Martedì · Rizzi, Campana · Archivio

Testo dell'archivio precedente (versione Codex), importato così com'è. Il giro automatico lo sostituirà con la nuova versione completa di video, quiz e flashcard.

In breve

Le due registrazioni disponibili e le slide Canvas integrate: spiegazioni, glossario e sei esercizi con feedback. Report NotebookLM richiesto, verifica ancora aperta.

Confine delle fonti. Il fascicolo usa 702 segmenti della registrazione mattutina, 895 della registrazione pomeridiana e il PDF Canvas *Lesson 3* di 55 pagine. I timestamp `R` e `C` sono offset nei due file, non orari del calendario. La parte orale del mattino copre ML e LLM; la parte pomeridiana è soprattutto laboratorio e conversazioni, non una lezione frontale su tutte le slide. In particolare il capitolo **data lake** è spiegazione del PDF, senza attribuzione orale verificata. Nessun file prova da solo ogni minuto della giornata.

1. Dal problema d'impresa al task di machine learning

Un'azienda di assistenza riceve reclami e vuole rispondere prima. «Usiamo AI» non definisce che cosa migliorare. La slide 10 chiede di tradurre il problema in un compito sui dati; le slide 6–8 usano **Task–Performance–Experience (T–P–E)**. *Task* è l'azione da apprendere, *Performance* è come misurarla su casi nuovi, *Experience* sono i dati o le interazioni da cui il sistema impara (R 19–28′). Per i reclami: T = assegnare una categoria; P = quota corretta e costo dei reclami prioritari classificati male; E = reclami storici con etichette controllate. Se il team valuta soltanto sui messaggi già usati per addestrare, la metrica può premiare la memoria invece della capacità di generalizzare.

La programmazione tradizionale esegue regole scritte su dati; ML stima un modello da esempi e feedback (slide 5). È una semplificazione didattica: anche un sistema ML contiene regole, controlli e software scritto da persone. Le slide 4 e 9 suggeriscono ML quando la regola è difficile da scrivere, i pattern sono apprendibili, il task ricorre e gli errori hanno un costo gestibile. L'esempio orale della lotteria pone il limite: se le estrazioni sono indipendenti, la storia non fornisce un segnale affidabile per la prossima. Un algoritmo più complesso non crea informazione assente.

Esempio ipotetico svolto. Una linea telefonica vuole prioritizzare ticket. Una baseline semplice ordina per parole chiave e ora di arrivo. Un modello ML usa testo e categorie storiche. La valutazione separa tickets successivi nel tempo, misura ritardi delle richieste urgenti e costi di falsi allarmi, poi verifica che un operatore possa correggere la priorità. Se i ticket urgenti sono rari, la sola accuratezza totale nasconde gli errori che contano.

Trasferimento. Il modello classifica il 95% dei ticket correttamente: puoi adottarlo? **Non ancora:** chiedi quanti urgenti perde, su quali casi è stato testato e che cosa accade dopo la classificazione.

Disegnare una valutazione che assomigli all'uso futuro

Performance non significa scegliere la metrica più alta nel notebook. Prima si scrive l'azione: chi riceve la previsione, quando, che cosa può fare e quale errore costa di più. Poi si costruisce un test che somigli al futuro. **Esempio ipotetico svolto:** i ticket storici del 2025 addestrano il modello, quelli di settembre 2026 lo valutano. Se le categorie di settembre sono state cambiate dall'azienda, l'accuratezza apparente può diminuire per una definizione diversa; serve una mappa delle categorie. Se si dividono a caso ticket duplicati tra training e test, il modello può memorizzare testi quasi identici e sembrare migliore di quanto sarà su reclami nuovi. Una baseline per parole chiave e un operatore esperto sono confronti utili prima di comprare infrastruttura ML.

Quando ML non è la prima risposta. Se l'azienda deve applicare una regola deterministica e stabile, per esempio calcolare l'IVA da un'aliquota ufficiale, una funzione controllata è più verificabile. Se i dati sono troppo pochi, sbagliati o privi di segnale, il modello non inventa la relazione. Se nessuno può agire sull'output, la previsione ha scarso valore operativo. **Trasferimento:** il task è «prevedere il prossimo numero di una lotteria corretta». Quale requisito manca? Un pattern apprendibile che dia risultati migliori della base casuale su estrazioni future.

Dalle competenze al risultato verificabile

Solo supporto della slide 11: SQL serve a interrogare ed estrarre dati; Python a scrivere trasformazioni e logica riproducibile; presentazione e visualizzazione a rendere comprensibile una decisione; ML a stimare o classificare quando gli esempi lo consentono; cloud services a lavorare sull'infrastruttura adottata; librerie deep learning/LLM a trattare rappresentazioni di testo, immagini o video con modelli compatibili. Non sono sei certificati da possedere prima di poter fare una domanda, né ogni problema richiede tutti gli strumenti.

Esempio ipotetico svolto: per ridurre consegne in ritardo, una query estrae ordini e date; una trasformazione controlla tipi e duplicati; una visualizzazione mostra ritardi per periodo; solo dopo si valuta se un modello migliora una baseline. La presentazione specifica decisione, metrica, limiti e

responsabile. Feedback: imparare un nuovo tool dimostra che il processo è migliorato? No: misura il risultato su casi nuovi e verifica che chi decide possa usare l'informazione. La competenza tecnica e quella di dominio si completano.

Quando il machine learning è giustificato

Fonte slide 4. La slide elenca quattro situazioni: expertise umana assente, expertise difficile da spiegare, modello da personalizzare e quantità enorme di dati. Sono ragioni per valutare ML, non quattro prove che ML funzionerà.

Come funziona. Se manca una regola esperta scrivibile, si cercano esempi e un risultato verificabile; se l'esperto sa agire ma non sa articolare ogni passaggio, si confronta il modello con il suo giudizio su casi nuovi; se la risposta cambia da utente a utente, si usano dati pertinenti e controlli per ciascun gruppo; se i dati sono massivi, servono metodi che estraggano pattern senza leggere a mano ogni riga.

Esempio ipotetico svolto. Un servizio clienti non riesce a scrivere regole per tutti i modi in cui si descrive un guasto. Raccoglie ticket già classificati, allena un classificatore, poi lo prova su ticket successivi. Per personalizzare la priorità, verifica separatamente che i clienti con contratto urgente siano riconosciuti senza far sparire gli altri. Il grande archivio consente il test, ma non dimostra qualità delle etichette.

Limite. Il caso "navigating on Moon" e gli altri esempi nella slide illustrano situazioni, non risultati misurati. Dati enormi ma non pertinenti o assenza di feedback affidabile possono rendere inadatto ML; una regola deterministica rimane preferibile quando basta.

Richiamo attivo. Quale delle quattro situazioni rende ML automaticamente necessario? **Feedback:** Nessuna. Traduci il caso in task, metrica, dati disponibili e costo degli errori; confronta una baseline semplice.

Parsimonia: la soluzione più semplice che regge la prova

Spiegazione didattica aggiunta sul nucleo riconoscibile nella trascrizione Rizzi, segmenti 125; le formulazioni ASR incerte restano non ricostruite.

Il richiamo ASR al rasoio di Occam permette una spiegazione didattica della parsimonia: fra alternative che spiegano adeguatamente gli stessi dati e reggono un confronto su casi nuovi, preferire quella con meno complessità non necessaria. Questo non garantisce verità o superiorità della soluzione più semplice. Una regola breve può essere più facile da controllare, mantenere e correggere; un modello più complesso può però risultare necessario se la regola perde i casi importanti.

Esempio ipotetico: una regola per parole chiave e un modello più grande trovano entrambi 90 dei 100 reclami urgenti nuovi, con gli stessi falsi allarmi. Se la regola costa meno da gestire, resta stabile ed è più trasparente, è una candidata ragionevole. Se il modello trova casi importanti che la regola perde, confronta quel beneficio con costi, rischi e manutenzione. Servono lo stesso test e gli stessi errori rilevanti, non soltanto la stessa accuracy totale.

Feedback: meno parametri significa automaticamente una soluzione migliore? No: la semplicità è una preferenza condizionata all'adeguatezza e alla prova.

---

2. Quattro modi di apprendere: chiedi da dove arriva il feedback

Nel **supervised learning** ogni esempio ha una risposta desiderata (*label*): foto con categoria oppure email marcate spam/non spam. Il modello impara la relazione fra segnali e label, poi si verifica su esempi nuovi (slide 12–14; R 28–30′). Se le label sono incoerenti, impara l'incoerenza. Nell'**unsupervised learning** non ci sono risposte assegnate: il sistema cerca somiglianze o gruppi (*clusters*) (slide 15–16; R 39–41′). Il cluster dipende dalle caratteristiche scelte e non è una «persona tipo» naturale o una causa.

Nel **semi-supervised learning** poche label orientano molti esempi non etichettati (slide 17; R 41–43′). Può ridurre il costo dell'annotazione, ma amplifica errori se le poche label non sono attendibili. Nel **reinforcement learning** un agente sceglie azioni in sequenza e riceve ricompense (*rewards*) per imparare una strategia (slide 18–19; R 30–33′ e 43–46′). Il punteggio deve corrispondere all'obiettivo: premiare solo la velocità può indurre un robot a ignorare sicurezza o qualità.

Esempio ipotetico svolto. Un negozio ha 100.000 recensioni e 500 valutate manualmente per tono: semi-supervised può usare entrambe, dopo controllo delle 500. Per scoprire temi senza categorie iniziali, prova unsupervised e chiedi a una persona di interpretare i gruppi. Per prevedere se una recensione è un reclamo con etichette affidabili, supervised. Per imparare a scegliere una sequenza di sconti in un ambiente simulato con feedback sulle vendite, reinforcement, ma prima occorre una simulazione valida e un limite alle perdite.

Trasferimento. Un gruppo di clienti emerso da clustering va automaticamente chiamato «alto valore»? **No:** la distanza matematica può riguardare età o frequenza d'uso; il valore commerciale richiede definizione, margine, controlli e prova su nuovi dati.

Dalla label alla ricompensa: cosa può andare storto

Nel **supervised** il caso difficile è la label. Se due operatori danno categorie diverse allo stesso reclamo, il modello impara un confine instabile. Occorre definire le categorie, controllare disaccordi e separare train e test. Nell'**unsupervised**, l'algoritmo non conosce l'obiettivo aziendale: può raggruppare clienti per volume d'acquisto, lingua o giorno della settimana a seconda di quali caratteristiche e scale riceve. Normalizzare variabili e interpretare i cluster è parte del lavoro; non si può chiamare «segmento fedele» un gruppo senza verifica della fedeltà.

Nel **semi-supervised** le poche label possono orientare tanti dati, ma se rappresentano soltanto clienti storici e il prodotto cambia, la pseudo-etichettatura può rafforzare il vecchio bias. Nel **reinforcement**, la ricompensa arriva dopo azioni in sequenza; un agente può imparare scorciatoie indesiderate se il punteggio non include sicurezza e costi. **Esempio ipotetico:** a un sistema per ottimizzare consegne si dà reward solo per velocità. Può scegliere percorsi non ammessi o penalizzare

zone difficili. Aggiungi vincoli legali, tempi massimi e metrica di servizio per tutti i clienti, poi valuta in simulazione prima dell'uso reale. **Trasferimento:** pochi esempi corretti più milioni senza risposta bastano per dire «semi-supervised è migliore»? No: confronta con supervised semplice, qualità delle label e prestazione su un test indipendente.

Nomi vicini non indicano altri quattro metodi

La slide 20 colloca **data mining** (ricerca di pattern nei dati), **Knowledge Discovery in Databases (KDD)** (processo più ampio che include preparazione, scoperta e uso), **data analysis** (esame dei dati per una domanda) e **pattern recognition** (riconoscimento di strutture o classi) vicino al ML. Sono tradizioni e ambiti con sovrapposizioni, non quattro nuove fonti di feedback accanto a supervised, unsupervised, semi-supervised e reinforcement. **Esempio ipotetico:** cercare gruppi ricorrenti nei reclami può essere data mining con clustering unsupervised; inserirli in un processo con pulizia, revisione umana e aggiornamento è una parte di KDD. **Trasferimento:** «data mining» ti dice da solo se sono presenti label? No: devi guardare il task concreto.

Regression, classification e aggiornamento bayesiano

Spiegazione aggiunta al richiamo ASR R 365–373,46:35–47:35. Regression, nel senso predittivo introduttivo, stima un valore numerico; classification assegna una classe o probabilità di classe. La famiglia di learning dipende dal feedback disponibile, non soltanto dal nome dell'algoritmo. Esempio ipotetico: stimare 18 minuti di consegna è un'uscita numerica; assegnare puntuale/in ritardo è una classe, definita da una soglia e da una scadenza. Puoi trasformare il primo risultato nel secondo solo specificando la regola. Valuti errori in minuti per la stima, errori di classe e loro conseguenze per la decisione. Una cifra molto precisa non rende precisa la previsione.

Bayesian methods indicano un modo di aggiornare una probabilità iniziale (prior) con nuova evidenza usando un modello di come quell'evidenza si genera, ottenendo una probabilità aggiornata (posterior). Non sono automaticamente sinonimo di supervised learning. Esempio didattico ipotetico: su 100 componenti,10 difettosi e 90 integri; un controllo segnala 8 dei 10 difettosi e 9 dei 90 integri. Fra i 17 segnalati,8 sono davvero difettosi:$8/17 \approx 47{,}06\%$. Non confondere questa quota con 8/10, che misura quanti difetti il controllo rileva. Il risultato dipende dalle frequenze e dall'attendibilità del controllo: non è una prestazione di un sistema reale. Feedback: la parola regression impone sempre una classe? No: specifica target e output. I riferimenti storici al programma lunare e la formulazione ASR deformata non sono adottati come storia verificata.

Matrix completion: riempire celle mancanti con cautela

Fonte slide 17. La slide presenta una matrice utenti × film con rating osservati e celle vuote per la raccomandazione. Matrix completion stima preferenze non osservate sfruttando regolarità fra utenti e film; una cella stimata non è un voto espresso.

Come funziona. Righe = utenti, colonne = film, celle note = valutazioni raccolte. Un modello cerca somiglianze e fattori latenti nei rating disponibili; usa queste strutture per stimare celle mancanti. La slide colloca l'esempio nel semi-supervised, ma la raccomandazione per rating sparsi non coin-

cide sempre con la definizione semplice “poche label + molti esempi senza label”: occorre specificare quale feedback è osservato.

Esempio ipotetico svolto. Ada valuta A=5 e B=4; Bruno valuta A=5, B=4 e C=5. Un sistema potrebbe stimare che Ada gradisca C e mostrarlo fra i suggerimenti. La stima va verificata con una risposta successiva di Ada o con un test su rating tenuti fuori dall’addestramento.

Limite. Rating mancanti non significano disinteresse: l’utente potrebbe non aver visto il film. Utenti o film nuovi, gusti che cambiano e popolarità sbilanciata rendono la stima fragile.

Richiamo attivo. Nella matrice, che cosa significa una cella vuota? **Feedback:** Preferenza non osservata, non “zero stelle”. Se il modello la riempie, è una previsione da valutare.

Data science e analytics: nomi del lavoro, non tipi di label

Fonte slide 20. La slide accosta ML, data mining/Big Data, KDD, data analysis, pattern recognition, data science e predictive/business/data analytics, con enfasi storiche diverse. Data science mette insieme domande, dati, metodi statistici, verifica e comunicazione; predictive analytics usa dati per stimare un esito futuro; business analytics lega l’analisi a una decisione d’impresa. Questi termini si sovrappongono e non identificano da soli un algoritmo.

Come funziona. Una stessa pipeline può includere preparazione dei dati (KDD), ricerca di pattern (data mining), stima di un ritardo (predictive analytics) e decisione sul personale (business analytics). Il rigore richiede confronto con baseline, test su dati nuovi e definizione della conseguenza operativa.

Esempio ipotetico svolto. Un corriere analizza consegne passate. Il data scientist definisce target e test temporale; il modello stima minuti di ritardo; il responsabile confronta un avviso automatico con la regola precedente e decide se usarlo. Il nome “analytics” non sostituisce nessuno di questi controlli.

Limite. La slide offre distinzioni storiche ed enfasi, non confini accademici universali; “predictive” non implica causalità e “business” non implica valore dimostrato.

Richiamo attivo. Se un team dice “facciamo data science”, sai se usa label? **Feedback:** No. Chiedi la domanda, l’output, il feedback disponibile e la prova sulla decisione reale.

Obiettivi supervisionati e strutture non supervisionate

Fonte slide 21. La figura oppone regression e classification con target noto a cluster analysis, density estimation e la dicitura “density reduction: select relevant variables” senza target dichiarato. Regression stima un numero, classification una classe. Cluster analysis raggruppa osservazioni simili; density estimation approssima dove i dati sono frequenti o rari nello spazio delle variabili. La voce “density reduction” della figura descrive selezione di variabili rilevanti: va letta come etichetta della slide, distinta dalla dimension reduction della slide successiva.

Come funziona. Senza label, gruppi e densità vengono ricavati dalle caratteristiche scelte; una zona poco popolata può segnalare casi insoliti, ma “raro” non equivale a frode o errore. Selezionare variabili significa decidere quali campi conservare, non comprimere necessariamente più campi in nuove componenti.

Esempio ipotetico svolto. Su 1.000 ticket con lunghezza del testo, ora e categoria storica, regression può stimare minuti di attesa usando un target numerico; classification può prevedere urgente/non urgente usando label. Senza categoria, clustering può proporre gruppi di testo; density estimation può mostrare che ticket notturni molto lunghi sono poco comuni, da leggere manualmente.

Limite. La figura non nomina anomaly detection. Non trasformare la rarità in diagnosi automatica; scale e variabili cambiano la struttura trovata.

Richiamo attivo. Quale metodo richiede una risposta corretta già nota per ogni esempio di training? **Feedback:** Regression/classification supervisionate nella figura. Clustering e stima di densità cercano struttura senza quel target.

Riduzione della dimensionalità e cautela su LDA

Fonte slide 22. La figura elenca PCA e Linear Discriminant Analysis (LDA) sotto “dimension reduction” nel riquadro unsupervised. Ridurre dimensionalità significa rappresentare molte variabili con meno coordinate, cercando di conservare l’informazione utile. PCA può trovare componenti dai pattern di variazione senza label. LDA, nel significato di Linear Discriminant Analysis qui scritto, usa le classi note per trovare direzioni che le separano: è una riduzione supervisionata, non un esempio generalmente unsupervised.

Come funziona. PCA combina variabili correlate in componenti; LDA calcola separazioni rispetto a classi note. Dopo la trasformazione si prova se un modello o un grafico risponde meglio alla domanda, confrontando anche la versione non ridotta. Chiarimento didattico esterno, non frase attribuita al docente: Scikit-learn §1.2.1.

Esempio ipotetico svolto. Un team ha 30 misure di utilizzo per ticket. Prova PCA per visualizzare i ticket su due assi senza usare categorie. Se invece ha categorie verificate e prova LDA, le usa per cercare assi che le separino. Valuta su ticket nuovi se la rappresentazione aiuta davvero.

Limite. Due assi possono perdere segnali importanti. La posizione di LDA nel riquadro della slide è una semplificazione o discrepanza: non va insegnata come regola.

Richiamo attivo. PCA e LDA in questa slide usano sempre lo stesso feedback? **Feedback:** No: PCA può usare dati senza classi, mentre LDA discriminante richiede classi note.

Leggere la tabella supervised/unsupervised riga per riga

Fonte slide 23. La tabella visiva ha cinque righe: objective, model training, model use, examples of algorithms, examples of applications. Nel suo esempio clinico mostra etichette sepsis/not sepsis a

sinistra e due phenotype a destra. A sinistra il training apprende una mappa fra input e label, poi in uso predice una label per un nuovo caso. A destra il training cerca sottogruppi omogenei senza quella label di uscita, poi assegna un nuovo caso a una struttura trovata. La riga algoritmi elenca logistic regression, gradient boosting e deep neural network a sinistra; K-means, latent class analysis e dimensionality reduction a destra. Le applicazioni elencate sono esempi della slide, non verifica di validità clinica.

Come funziona. Per usarla, leggi ogni colonna dall'obiettivo al tipo di feedback e infine all'output; non dedurre che il nome di un algoritmo garantisca la correttezza dell'applicazione. Un gruppo non supervisionato ha bisogno di interpretazione e validazione esterna.

Esempio ipotetico svolto. Scenario non clinico: 200 reclami sono etichettati "consegna"/"pagamento", altri 800 no. Con label controllate, un classificatore prova ad assegnare le due categorie a reclami futuri e si valuta contro un test etichettato. Senza tali label, un clustering trova gruppi di testi; un operatore controlla se i gruppi hanno senso e se cambiano con parole o scale diverse.

Limite. L'esempio sepsis/phenotype è soltanto la figura originale e non fornisce consigli diagnostici o risultati misurati. La colonna unsupervised non prova che i gruppi siano "veri" tipi naturali.

Richiamo attivo. Quale riga della tabella ti dice se puoi misurare errori contro una risposta nota? **Feedback:** Model training, insieme all'obiettivo: a sinistra ci sono label note; a destra la struttura è cercata e richiede altri controlli.

---

3. AI, ML, deep learning e generative AI non sono sinonimi

La lezione colloca il machine learning dentro l'AI e il deep learning come una famiglia di ML; i **Large Language Models (LLM)** usano modelli profondi per trattare e generare sequenze linguistiche (slide 24–35; R 50–62′). La classificazione delle email, la previsione di un ritardo e la generazione di una risposta sono task differenti. Un testo ben scritto non convalida il numero previsto. L'AI comprende anche metodi non generativi; una presentazione in cui compare un chatbot non rende «LLM» ogni algoritmo di previsione.

Le slide 28–30 contrappongono un **decision tree** che percorre condizioni leggibili (colore, forma, consistenza di una mela) a una **neural network** di connessioni pesate. Il tree può mostrare un percorso decisionale, ma può comunque essere instabile o errato. Una rete rappresenta relazioni complesse, ma i suoi pesi non corrispondono necessariamente a concetti umani puliti come «colore» in un livello e «forma» nel successivo. L'analogia del cervello è didattica, non un'identità biologica.

Nel parlato compaiono **induzione** (da casi osservati a regolarità), **abduzione** (proporre una spiegazione plausibile di un effetto) e **deduzione** (applicare premesse a un caso) (R 52–55′). Esempio ipotetico: osservo che i reclami aumentano dopo consegne tardive e induco una relazione da testare; ipotizzo con abduzione che un nuovo corriere sia una possibile causa; deduco che, se il contratto prevede un rimborso per ogni ritardo verificato, il caso X richiede il rimborso. Né induzione né abduzione sono prova di causalità senza controlli.

Le slide 12 e 26–27 introducono **Absolute Zero Reasoner** come frontiera di generazione e verifica di task. Il paper Absolute Zero: Reinforced Self-play Reasoning with Zero Data descrive task di codice auto-generati con un executor che verifica e fornisce reward. «Zero data» nel titolo non significa assenza di pretraining né capacità generale autonoma. Questo chiarimento è aggiunto alla spiegazione, non attribuito come formulazione esatta della voce PLAUD.

Trasferimento. Un LLM produce un avviso ai clienti sui ritardi: come testi il sistema? **Risposta:** verifica separatamente la previsione del ritardo sui dati reali e la fedeltà del messaggio a ordine, data e servizio; la fluidità della frase non è una metrica di correttezza.

Albero e rete sulla stessa mela

Un **decision tree** può dire: «se forma ovale e consistenza liscia, poi guarda il colore». Il percorso rende visibile quali condizioni hanno portato all'etichetta, ma l'albero può sovradattarsi a esempi piccoli o usare un indizio spurio. Una **neural network** trasforma segnali numerici attraverso livelli di pesi. Può rappresentare confini più complessi, ma una risposta «mela: 0,8» non rivela automaticamente quale caratteristica l'abbia causata. La probabilità prodotta va inoltre calibrata prima di trattarla come 80% di certezza nel mondo reale.

Esempio ipotetico svolto: foto di mele in training sono tutte su sfondo bianco, foto di pere su sfondo nero. Entrambi i modelli potrebbero imparare lo sfondo, non il frutto; una rete più complessa può persino mascherare l'errore meglio. Testare su nuove foto con sfondi variati rivela il problema. **Trasferimento:** un modello accurato in laboratorio riconosce male frutti al supermercato. Quale ipotesi controlli? Cambiamento di luce, sfondo, varietà e fotocamera prima di dichiarare che «ML non funziona».

Symbolic AI e metodi che apprendono

La tassonomia visiva della slide 25 ricorda che l'AI comprende **symbolic AI**, dove regole e rappresentazioni esplicite permettono inferenze, e metodi che apprendono da dati. Una regola «se la domanda è arrivata entro il termine, accetta il modulo» può essere simbolica; una classificazione dei reclami da esempi è learning-based. I due approcci possono convivere: un modello legge un documento, una regola applica una scadenza ufficiale e un operatore controlla eccezioni. Un LLM è un sottoinsieme dei modelli generativi, non il contenitore di tutto il campo. **Trasferimento:** se una norma cambia, basta riaddestrare un modello? No: la regola ufficiale va aggiornata e verificata; il modello può aiutare a leggere testo ma non sostituisce l'autorità della fonte.

Absolute Zero: seguire i cinque controlli del ciclo

La figura Canvas p 27 descrive un modello già preaddestrato con due ruoli, proposer e solver. (1) Propose: costruisce un compito di ragionamento su codice. (2) Validate: un esecutore controlla che il compito sia valido. (3) Solve: il ruolo solver tenta una risposta. (4) Verify and reward: l'esecuzione confronta la risposta con un risultato verificabile e fornisce feedback. (5) Update and repeat: l'apprendimento per rinforzo aggiorna i ruoli e orienta i compiti successivi. Un compito sintatticamente valido non è ancora una risposta corretta; una risposta corretta una volta non dimostra ra-

gionamento universale. Zero riguarda il dataset di ragionamento curato manualmente durante quel self-play, non assenza di pretraining, dati, ambiente o regole.

Esempio didattico della figura: f(x)=2 x+1. Deduction: dati programma e input 3, prevedi output 7. Abduction: dati programma e output 7, trova input 3 valido. Induction: dagli esempi 1→3,2→5,3→7, proponi un programma compatibile. Nel terzo caso più programmi possono adattarsi ai pochi esempi: occorrono prove ulteriori e un criterio di verifica; non si deduce una legge unica da tre coppie. Feedback: se il solver scrive 8 nel primo caso, il verificatore restituisce un errore; se propone un compito senza esito valutabile, il controllo deve fermarlo prima del premio. Questo illustra la figura, non è un training eseguito qui né una promessa di capacità generali.

Backprop, aggiornamento dei pesi e uso del modello

Spiegazione didattica aggiunta sul nucleo riconoscibile nella trascrizione Rizzi, segmenti 444; le formulazioni ASR incerte restano non ricostruite.

Backpropagation, o backprop, calcola come una funzione di errore, la loss, cambia rispetto ai parametri di una rete neurale, riportando il calcolo delle derivate attraverso i suoi livelli. Un algoritmo di ottimizzazione usa queste informazioni per scegliere un aggiornamento dei pesi: calcolo dei gradienti e scelta dell'aggiornamento sono passaggi distinti. La loss misura uno scostamento dall'obiettivo scelto; un reward è invece un feedback sull'esito di azioni. Non sono sinonimi e ricevere una ricompensa non prova da solo che sia stato eseguito backprop. Alcuni sistemi di reinforcement learning usano reti e metodi a gradiente; non ogni algoritmo ML o ogni procedura di training deve usare questa tecnica.

Esempio ipotetico: una stima è 1 e il target è 2. Con loss quadratica = (previsione − target)², l'errore è 1. Se dopo un aggiornamento la stima è 1,5, sullo stesso caso la loss è 0,25. Questo non prova un miglioramento sui casi nuovi, né garantisce che ogni passo riduca ogni errore. Durante l'inference si usano parametri appresi per produrre un output; un nuovo prompt non dimostra che i pesi siano automaticamente riaddestrati.

Feedback: una risposta corretta prova un nuovo apprendimento dei pesi? No: può derivare da parametri già appresi. La spiegazione chiarisce il termine ASR senza attribuire un algoritmo preciso al passaggio deformato.

Chiarimento tecnico esterno: Scikit-learn: neural networks, riletto il 19 ottobre 2026; non prova quale algoritmo fosse inteso nel segmento ambiguo.

---

4. Come un LLM produce testo e perché può sbagliare

La **tokenization** divide testo in unità che possono essere parole, parti di parola o simboli. Il modello usa contesto e parametri appresi per attribuire probabilità ai possibili token successivi e genera una sequenza (R 63–79′; slide 31–35). Un **Transformer** usa meccanismi di attention per combina-

re informazioni della sequenza; l'articolo Attention Is All You Need del 2017 è un chiarimento esterno, non una citazione del docente. La **temperature** modifica la distribuzione con cui si selezionano i token: aumentando la variabilità possono emergere più alternative e più errori. Non trasforma il modello in motore di fact-checking e non fornisce una probabilità calibrata che una frase sia vera.

La metafora orale del «database di probabilità» aiuta a immaginare la generazione, ma un LLM non è un archivio di record verificati interrogabile come una tabella SQL. Può produrre una risposta plausibile e falsa, soprattutto per date, numeri, citazioni e casi non presenti o ambigui nel contesto. Quando si usa per studiare, l'output è una bozza da confrontare con slide, trascrizione corretta e fonti originali. Il dibattito **weak AI / strong AI** sull'«understanding» non è risolto dalla fluidità del testo (R 80–86′).

Esempio ipotetico svolto. Chiedi a un LLM «quanti studenti hanno superato l'esame?». Risponde 85% con una frase sicura. Senza registro e denominatore, non puoi sapere se il numero è reale, quale coorte riguarda o se include assenti. La verifica è recuperare fonte, periodo e definizione, non abbassare o alzare la temperature finché la frase convince.

Trasferimento. Il modello cita una pagina 80 di un PDF di 55 pagine. Quale parte del processo fallisce? **Risposta:** la verifica della fonte. Occorre aprire il PDF originale, correggere o scartare la citazione e non insegnare la frase come fatto.

Il percorso token → risposta, senza magia

Prendi la frase «Il contratto scade domani». La tokenizzazione potrebbe spezzare parole frequenti o rare in unità diverse: il numero di token non coincide stabilmente col numero di parole. L'input viene trasformato in rappresentazioni numeriche; i livelli del modello usano il contesto per produrre una distribuzione dei token successivi. Se genera «domani», il modello non ha consultato per forza il contratto reale: ha scelto una continuazione plausibile del contesto dato. Un sistema con recupero di documenti può aggiungere una fonte, ma va controllato che la risposta usi il documento giusto e non inventi una pagina.

Temperature e verificabilità. A temperatura più bassa l'uscita tende a concentrarsi su token più probabili; a temperatura più alta tende a esplorare alternative. Non esiste un valore che trasformi il testo in fatto. **Esempio ipotetico:** chiedi tre slogan per un corso: più varietà può essere utile. Chiedi la scadenza ufficiale di un esame: serve la pagina Canvas corrente e una citazione verificata, non creatività. **Trasferimento:** due risposte identiche sono più vere di due risposte diverse? Non necessariamente: ripetibilità e verità sono proprietà distinte.

Generare testo, immagine o audio richiede input e rappresentazioni compatibili

La slide 35 raggruppa testo, immagini, video e audio fra i possibili output della generative AI. Il punto comune è apprendere regolarità dai dati e produrre un output condizionato da un input; la rappresentazione concreta può cambiare: token testuali, rappresentazioni dell'immagine, segmenti

o rappresentazioni del segnale audio. Non ogni LLM accetta o produce ogni modalità, e non tutto il pensiero umano si riduce per questo a linguaggio. La lista storica di prodotti della slide non prova funzioni o disponibilità attuali di ciascuno.

Esempio ipotetico: un'impresa vuole descrivere un difetto in una foto e produrre un avviso sonoro. Prima verifica se il modello può leggere l'immagine; poi controlla la descrizione sui pixel originali; infine usa una funzione compatibile con l'output audio e ascolta l'avviso. Tre verifiche distinte: la fluidità del testo non prova che la foto sia stata letta o che l'audio sia corretto. Feedback: una chat solo testuale che riceve il nome di un'immagine ha visto il difetto? No: serve accesso effettivo alla modalità e una prova sul contenuto. Le metafore ASR R 655–661 non certificano quella capacità.

Risorse del sistema e unità di utilizzo

Spiegazione didattica aggiunta sul nucleo riconoscibile nella trascrizione Rizzi, segmenti 528, 602, 698; le formulazioni ASR incerte restano non ricostruite.

Costruire e far funzionare un modello richiede risorse: hardware o infrastruttura, lavoro tecnico, manutenzione ed energia. Il costo di accesso o consumo di un servizio non coincide automaticamente con il costo fisico di una risposta. Quando un servizio misura l'uso in token, le condizioni possono distinguere ingresso e uscita; altri accessi possono includere canoni o limiti. Non esiste un rapporto fisso fra parole e token.

Esempio ipotetico di pianificazione: 100 documenti, ciascuno con 500 token in ingresso e 100 in uscita, producono 50.000 token di ingresso e 10.000 di uscita. Queste sono unità di utilizzo, non prezzi né una misura di energia. Per stimare un costo monetario servirebbero le condizioni effettive applicabili e le eventuali componenti fisse. Confronta alternative sullo stesso compito mantenendo qualità, verifica delle fonti e lunghezza utile dell'output: una risposta sbagliata può aggiungere lavoro di correzione.

Feedback: ridurre le parole del 10% garantisce una bolletta o un consumo energetico inferiori del 10%? No: tokenizzazione, componenti fisse, modalità d'uso e infrastruttura impediscono quella deduzione automatica. Il testo ASR non documenta tariffe, dimensioni o consumi di un servizio reale.

---

5. Data lake: contenuto del PDF, copertura orale non attestata

Le slide 36–48 presentano il **data lake** come deposito capace di conservare dati strutturati, semistrutturati e non strutturati spesso nel formato nativo. La promessa è tenere materiale per analisi diverse; il rischio è creare un deposito impossibile da capire, controllare o usare. La catena comprende **ingestion** (entrata batch o stream), **storage**, trasformazione, catalogazione e accesso per analisi. I **metadata** descrivono significato, unità, periodo e proprietario; **lineage** traccia da dove arriva un risultato e quali trasformazioni ha subito; **auditing** registra chi o che cosa ha cambiato un dataset; **security** applica autenticazione e autorizzazioni (slide 37–45). Le slide 46–48 aggiungono

fasi di maturità e pratiche architetturali. Questi elementi sono supportati dal PDF, non da un segmento orale identificato nelle due registrazioni.

Esempio ipotetico svolto. Un retailer deposita ordini, resi, immagini e log nello stesso lake. La dashboard dice che le vendite di un prodotto sono raddoppiate. Il lineage rivela che il nuovo import ha duplicato ogni ordine; l'audit indica quando; il catalogo mostra che una tabella usa importi lordi e l'altra netti. Il lake ha conservato i dati, ma la decisione sulle scorte è valida solo dopo deduplica, definizione coerente e permessi pertinenti. «Data democratization» non significa accesso indiscriminato.

Trasferimento. Se una tabella è visibile a tutto il team, è utilizzabile per qualunque analisi? **No:** servono significato, provenienza, qualità, scopo e autorizzazione; la disponibilità tecnica è un controllo diverso.

Cinque controlli che rendono il lake utilizzabile

Ingestion: registra fonte, versione, data di acquisizione e modalità batch/stream; una copia senza provenienza è difficile da correggere. **Catalogo e metadata:** descrivono significato di colonne, unità, proprietario, periodo e permessi; rendono il dato trovabile e interpretabile. **Qualità:** controlla mancanti, duplicati, coerenza delle chiavi e intervalli plausibili prima di collegare tabelle. **Lineage e auditing:** permettono di ricostruire trasformazioni e cambiamenti; se il KPI cambia, sai quale import o regola cercare. **Sicurezza:** limita accesso per ruolo e scopo, con traccia delle operazioni. Questi controlli si sostengono a vicenda: un dataset ben catalogato ma accessibile a tutti può violare la privacy; un dataset sicuro ma senza definizioni può produrre analisi sbagliate.

Le slide 46–48 descrivono la **maturità** come passaggio dall'ingestione a capacità analitiche e governance integrate; non sono una prova che una singola azienda attraversi automaticamente quelle fasi. **Esempio ipotetico svolto:** un report vendite combina un file lordo e uno netto. Il catalogo chiarisce le definizioni, il lineage mostra la trasformazione IVA, la qualità rileva ordini duplicati, i permessi impediscono accesso non necessario ai dati personali. Senza questi passaggi, il lake accumula dati ma non conoscenza affidabile. **Trasferimento:** un KPI cambia dopo un nuovo import; da dove parti? Controlla audit e lineage, poi confronta definizioni e righe, prima di raccontare una nuova tendenza di mercato.

Dato grezzo, dato pronto e controllo del cambiamento

Un lake può mantenere una zona **raw** per i file originali e una zona **curated** per dati puliti e definiti. Non si cancella il grezzo solo perché una trasformazione è stata eseguita: serve ricostruire da dove viene un risultato e correggere errori. Ma non si conserva neppure «tutto per sempre»: tempi di conservazione, permessi e costi devono seguire scopo e regole applicabili. **Esempio ipotetico:** il KPI dei resi cambia dopo che il team aggiorna la definizione di «reso concluso». Il lineage collega dashboard, tabella curata e file sorgente; l'audit mostra la data del cambio; una nota di versione consente di confrontare prima e dopo senza fingere che la serie sia omogenea. **Trasferimento:** due report usano lo stesso nome di KPI ma regole diverse. Quale è corretto? Non puoi decidere dal nome: servono definizione, periodo, fonte e decisione per cui viene usato.

Architettura, maturità e interfacce del data lake

Solo supporto Canvas pp 40–41, non parlato attestato. Ingestion acquisisce batch o flussi e registra la fonte; storage conserva dati accessibili: HDFS è la landing zone esemplificata dalla slide, non un obbligo universale. Distillation prepara e struttura ciò che serve all'analisi. Processing esegue query o algoritmi. Insights rende il risultato interrogabile, per esempio con SQL o strumenti di analisi. Unified operations governa monitoraggio, audit, workflow e gestione trasversalmente. Il termine proficiency management della slide non riceve una definizione tecnica dimostrata e resta un'etichetta del materiale. La figura distingue dati at rest, conservati, da dati trattati con bassa latenza: real time è una scelta di processo e tempi, non sinonimo di dato vero o aggiornamento istantaneo garantito.

Esempio ipotetico svolto: acquisisci ordini una volta al giorno per un report mensile; registra origine e chiavi, conserva il grezzo, normalizza date e valute, aggrega ricavo e pubblica il risultato con definizione e lineage. Un allarme che deve intervenire in pochi secondi richiede invece un flusso, tempi e controlli adatti. Feedback: rendere ogni passaggio real time migliora automaticamente il report? No: aumenta complessità e costi, mentre la decisione mensile potrebbe non richiederlo. Verifica latenza necessaria e affidabilità di ogni raccordo.

Le quattro fasi didattiche di pp 46–47 sono: acquisire dati su scala; costruire capacità di trasformazione/analisi; far lavorare EDW e lake insieme diffondendo risultati utili; integrare governance, lifecycle e metadata nell'impresa. I testi delle prime due fasi si sovrappongono nel PDF: la sequenza è una griglia, non una scala di certificazione o una statistica sulla maturità reale. Un retailer che salva file ma non ritrova il dato del report deve prima migliorare catalogo e trasformazioni; uno con lake e warehouse che definiscono revenue diversamente deve riconciliare significati e responsabilità. Più prodotti installati non provano uno stadio più alto.

Le buone pratiche di p 48 richiedono formati nativi, componenti sostituibili con API, gestione distinta di discovery, ingestione, storage, qualità, trasformazione e visualizzazione, adattamento al settore, ingresso rapido delle sorgenti e compatibilità con la gestione aziendale esistente. Schema-on-read significa applicare struttura e requisiti del dato quando lo si interroga o prepara per il consumo, anziché imporre un unico schema completo all'ingresso; non elimina validazione e qualità. API è il contratto fra componenti: campi, tipi, versioni, accessi ed errori. Esempio ipotetico: sostituisci il motore di grafici, ma mantiene la stessa definizione di ricavo e gli stessi permessi; verifica input, output, totali e gestione dei valori mancanti prima di adottarlo. Feedback: schema-on-read significa caricare qualunque dato senza scopo o diritti? No: flessibilità tecnica non elimina qualità, sensibilità, accessi o responsabilità. Un dominio sanitario e un catalogo pubblico di prodotti richiedono controlli differenti, non la stessa policy copiata senza analisi.

Flat architecture: ritrovare e governare ogni oggetto

Fonte slide 38. La slide descrive un data lake come storage economico e “flat”, con unique identifier e tag di metadata per ogni elemento; lo contrappone a un warehouse “hierarchical”. Nell’architettura descritta, l’oggetto è conservato con identificatore e metadata che rendono ricercabili fonte, data, formato, significato, proprietario e permessi. Il ricercatore trova un dataset dal catalogo inve-

ce di dover conoscere un percorso di cartelle. Le trasformazioni per consumo possono arrivare dopo l'ingestione.

Come funziona. Nel modello della slide, file diversi entrano come oggetti distinti. Un identificatore separa gli oggetti; tag e metadata descrivono fonte, periodo, formato, significato, proprietario e permessi. Il catalogo consente di trovarli; prima del consumo si verifica lo schema e si prepara una tabella con definizioni controllate.

Esempio ipotetico svolto. Un retailer carica un CSV ordini e immagini prodotto. A ciascun oggetto assegna un ID; nel catalogo registra data, fonte e responsabile. L'analista cerca "ordini settembre", controlla che il campo ricavo sia netto e crea una tabella curata per la dashboard. Se trova due versioni, confronta provenienza e trasformazioni prima di scegliere.

Limite. La contrapposizione "lake flat" e "warehouse gerarchico" è una semplificazione della slide, non una definizione universale dei warehouse moderni. ID e tag non attestano da soli qualità, correttezza o diritti d'uso.

Richiamo attivo. Se un file ha un ID univoco, sai già che il suo "ricavo" è netto? **Feedback:** No. L'ID identifica l'oggetto, mentre una definizione verificata chiarisce se il ricavo è netto o lordo. L'alternativa "ID uguale significato" confonde rintracciabilità e interpretazione del dato.

Le promesse del data lake richiedono prove

Fonte slide 39. La slide propone raw data, storage facilitato (cita Hadoop), agilità, qualità delle analisi, ML/AI, assenza di silos, vista cliente a 360° e vantaggio competitivo. Conservare dati diversi può evitare una modellazione completa all'ingresso e accelerare esperimenti, se catalogo, diritti e costi funzionano. Più volume può aiutare un modello solo quando aggiunge segnale pertinente e qualità. Integrare fonti può ridurre silos solo con identità, definizioni e permessi coerenti; la vista cliente è un risultato da costruire e verificare. Il vantaggio competitivo richiede una decisione migliore misurata, non la sola piattaforma.

Come funziona. Raw storage può ridurre il lavoro di modellazione iniziale; agilità richiede ricerca e accesso effettivi; ML richiede dati pertinenti e test su casi nuovi; meno silos richiede identità e definizioni coerenti fra fonti; una vista cliente ampia richiede copertura e diritti verificati. Il vantaggio competitivo emerge solo se queste capacità migliorano una decisione rispetto a una baseline, tenendo conto di costi e rischi.

Esempio ipotetico svolto. Un retailer unisce ordini e resi di settembre tramite una chiave cliente controllata. Elimina duplicati, verifica che "reso" significhi operazione conclusa e limita l'accesso ai campi personali. Addestra un modello di previsione dei resi sui mesi precedenti e lo valuta su ottobre contro la regola già usata dal magazzino. Solo se gli errori utili alle scorte diminuiscono abbastanza da compensare i costi, il caso sostiene un beneficio.

Limite. Hadoop è un esempio citato nella slide, non un requisito. Più volume non garantisce qualità; ML non rende automaticamente previsioni profittevoli. La "vista a 360°" può essere incompleta o non lecita per determinati dati.

Richiamo attivo. Quale prova manca fra “abbiamo caricato tutti i dati” e “abbiamo un vantaggio”? **Feedback:** Un confronto su decisioni e dati nuovi, con qualità, costi e diritti controllati. Il semplice caricamento dimostra capacità di storage, non un miglioramento misurato del risultato.

Discovery, tagging e accounting prima dell’analisi

Fonte slide 44. La slide richiede security in ogni layer, authentication, accounting, authorization e data protection, qualità e data discovery tramite tagging. Authentication verifica chi accede; authorization limita ciò che può fare; accounting registra/attribuisce l’uso delle risorse o le operazioni per controllo e responsabilità. Data discovery usa catalogo, tag e descrizioni per trovare e comprendere il dato prima di prepararlo o analizzarlo. I controlli di qualità evitano insight costruiti su duplicati o campi incoerenti.

Come funziona. Authentication verifica l’identità; authorization limita azioni e dataset; accounting attribuisce e registra uso o operazioni per controllo e responsabilità. Data discovery usa catalogo, tag e descrizioni per trovare e comprendere il dato prima della preparazione. La qualità controlla duplicati, mancanti e definizioni incoerenti prima di trarre insight.

Esempio ipotetico svolto. Una squadra cerca il dataset “resi” nel catalogo. Tag, periodo e proprietario portano a due tabelle; un campione mostra che una conta richieste aperte e l’altra resi conclusi. Per un report sui resi completati sceglie la seconda, registra la definizione e concede accesso soltanto al ruolo autorizzato. Il log attribuisce l’esportazione usata nel report.

Limite. Un tag può essere obsoleto; accounting non impedisce da solo un accesso improprio. Sicurezza, qualità e discovery vanno verificate lungo ingestion, storage e consumo, non date per buone perché un file è reperibile.

Richiamo attivo. Un analista trova un dataset taggato “vendite”: può pubblicarne subito un grafico? **Feedback:** No. Prima controlla definizione, periodo, qualità e permessi, poi conserva la traccia dell’uso. Il tag facilita la ricerca ma non certifica che il dato sia corretto o pubblicabile.

Exploration: scegliere il dataset prima del grafico

Fonte slide 45. La slide distingue auditing (chi/quando/cosa è cambiato), lineage (origine e trasformazioni) e data exploration come fase iniziale per identificare il dataset giusto. Exploration ispeziona schema, righe campione, copertura temporale, unità, mancanti e distribuzioni per decidere se il dataset risponde alla domanda. Auditing e lineage servono dopo e durante l’analisi per spiegare versioni e cambiamenti. Il catalogo restringe la ricerca; l’esplorazione verifica il contenuto reale.

Come funziona. Exploration ispeziona schema, campione, copertura temporale, unità, mancanti e distribuzioni per scegliere dati adatti alla domanda. Auditing registra chi/quando/cosa ha cambiato; lineage collega origine e trasformazioni al risultato. Catalogo ed exploration trovano e valutano il dataset; audit e lineage aiutano a spiegare versioni e cambiamenti.

Esempio ipotetico svolto. Per chiedere “sono aumentati i resi?”, un retailer trova due tabelle: richieste aperte e resi conclusi. Confronta definizioni, date e conteggi, sceglie la seconda se vuole mi-

surare operazioni completate ed esplicita l'esclusione delle richieste aperte. Controlla con lineage da quali file e regole deriva l'aggregato e con audit se la regola è cambiata nel periodo.

Limite. Una rapida anteprima può non rivelare errori rari o valori fuori periodo. L'esplorazione iniziale non sostituisce controlli di qualità, permessi e revisione del risultato finale.

Richiamo attivo. Qual è il primo controllo prima di usare la tabella chiamata "resi"? **Feedback:** Definire cosa rappresenta ogni record e verificare periodo, campi e copertura rispetto alla domanda. Il nome "resi" da solo può riferirsi a richieste o a operazioni concluse.

---

6. Il laboratorio: dashboard prima del pitch, domanda prima del grafico

Le slide 49–55 chiamano l'attività **Dashboarding Oscars**: scegliere un dataset, cercare una domanda, valutare qualità e campi, progettare pochi grafici, costruire una dashboard e spiegare l'insight in un breve pitch. La frase operativa è **curiosity before clicks**. Nella seconda registrazione una voce indica Databricks e chiede quale insight si cerchi (C 3–7′); più tardi chiarisce che il pitch parte dalla dashboard (C 45–47′). Compare anche una concessione situata a usare slide se una dashboard non viene completata (C circa 63′): non sostituisce la consegna generale senza verifica su Canvas. Una risposta nell'audio consente un CSV anche da fonte diversa da Kaggle (C 30–31′).

Esempio ipotetico svolto. Un gruppo trova dati pubblici su vendite musicali. Domanda: i brani più recenti hanno ascolti più alti nel campione? Prima controlla unità, data di raccolta, duplicati, piattaforma e scelta del campione. Un grafico scatter può mostrare relazione apparente, ma non dimostra che l'anno di uscita causi gli ascolti: promozione, artista, disponibilità e tempo di esposizione possono confondere. Il pitch corretto dice «nel campione osservato» e propone un controllo; non «abbiamo scoperto la legge del mercato».

I pitch su sonno, salute e wearable e le discussioni degli studenti nel file pomeridiano (C circa 70–90′) sono **proposte o dichiarazioni degli studenti**, non risultati con dataset e calcoli qui validati. Non si deducono identità dalle etichette `Speaker N`. L'importazione CSV → tabella → dashboard descritta intorno a C 58–60′ è una guida contestuale, non prova che ogni gruppo abbia completato o pubblicato l'asset.

Trasferimento. In un grafico paesi con più gioco d'azzardo hanno più famiglie monoparentali. Puoi dire che il gioco causa l'esito? **No:** definizioni nazionali, anni, popolazione, fattori socioeconomici e selezione dei paesi vanno controllati; una correlazione aggregata non prova causalità individuale.

Una dashboard è un argomento, non una collezione di grafici

Una domanda buona identifica fenomeno, unità, periodo e decisione: «quali corsi fitness hanno no-show più alto nelle ultime otto settimane, e quali orari proviamo a modificare?». Il **dataset** deve contenere prenotazioni, accessi, date e definizioni; un grafico a barre dei no-show per fascia oraria

può rendere visibile il problema. Ma se il registro accessi perde eventi serali, il grafico inganna. Il **pitch** deve dire che cosa si osserva, quale decisione si propone, quali limiti restano e quale prova successiva serve.

Esempio ipotetico svolto: un gruppo trova una correlazione fra energia e loudness in brani musicali. Prima controlla che entrambe le variabili abbiano definizione e scala comparabili e che il campione non contenga duplicati. Lo scatter può mostrare relazione, non dire che alzare loudness causa più energia percepita né che aumenta gli ascolti. Un pitch onesto presenta il pattern, il campione e una possibile verifica su brani fuori campione. Un pitch debole usa molti grafici senza una decisione o attribuisce causalità dalla sola correlazione.

Trasferimento: hai 40 minuti per preparare la dashboard. Quali tre azioni fai prima di scegliere i colori? Formuli la domanda, esamini qualità/limiti dei campi e disegni il grafico che rende giudicabile una risposta; soltanto poi sistemi estetica e interazioni.

Dalla tabella agli assi: controllare anche un risultato incerto

Parafrasi circoscritta del laboratorio C 765–788,67:12–68:33: si discute collegare una tabella alla dashboard, aggiungere dati e scegliere assi e visualizzazione. Non attribuiamo il completamento a un gruppo né garantiamo nomi di pulsanti attuali. Procedura didattica: controlla righe, tipi e chiavi dopo l'import; seleziona la tabella corretta; definisci la domanda; assegna X e Y con unità e periodo; stabilisci se mostrare righe o aggregati; confronta un valore del grafico col dato sorgente. Un campo numerico identificativo non diventa una quantità sommabile solo perché il software offre Sum.

Esempio ipotetico svolto: ogni riga è una prenotazione con data, corso e stato di presenza. Per confrontare no-show mensili, X è mese e Y può essere numero di assenze o percentuale assenze/prenotazioni: sono misure diverse. Se un corso ha 8 assenze su 10 prenotazioni e un altro 20 su 100, i conteggi indicano 20 contro 8, ma i tassi 20% contro 80%. Prima scegli la decisione, poi etichetta il denominatore. Feedback: un grafico bello con la somma del codice cliente risponde alla domanda? No: serve una misura coerente con l'unità di analisi, non un'aggregazione predefinita.

Nel dialogo C 817–836,74:50–77:26, alcuni studenti dicono di non trovare una relazione chiara o utile per la loro proposta. Il dataset e il calcolo non sono qui validati: non adottiamo l'esito come fatto. Il metodo da imparare è presentare anche un risultato nullo o incerto: descrivi campione, variabili e definizioni, mostra il grafico, spiega che cosa il controllo permette e quali limiti restano. Non inventare un nesso per rendere il pitch interessante; non dedurre neppure indipendenza o assenza di causalità da un grafico senza relazione evidente. Un pattern non lineare, pochi casi, misure rumorose o campioni diversi possono cambiare la lettura. Feedback:«non vedo una linea, quindi X non può influire su Y» va oltre la prova;«in questo campione e con questo controllo non emerge una relazione chiara» delimita il risultato.

La slide 51 nomina cinque premi della Dashboarding Oscars; le interpretazioni seguenti sono spiegazione didattica aggiunta, non una rubric formalizzata dal corso: design (leggibilità e facilità d'uso), deepest insight (una domanda con risposta sostenuta), data integration (sorgenti collegate senza alterarne significato), storytelling (percorso che rende rilevante il dato) e people's choice

(preferenza del pubblico). Un voto del pubblico non certifica la verità e un insight inatteso non giustifica cherry-picking. Esempio ipotetico: due sorgenti unite con una chiave duplicata producono una storia convincente ma un totale falso; il controllo delle chiavi viene prima della premiazione. Nel pitch separa osservazione, interpretazione, decisione e limite.

Ripasso a libro chiuso

1. Scrivi T–P–E per i reclami e un errore costoso. **Feedback:** un compito misurabile e una prova su casi nuovi sono più concreti di «fare AI».
2. Distingui le quattro famiglie di learning dal feedback disponibile. **Feedback:** label complete, nessuna label, poche label, reward di una sequenza.
3. Perché un LLM non è una fonte verificata? **Feedback:** genera testo condizionato dal contesto; una citazione deve essere controllata sulla fonte.
4. Perché il data lake non garantisce una dashboard giusta? **Feedback:** metadati, chiavi, qualità, lineage e diritti determinano se il dato è interpretabile e lecito.
5. Che cosa devi dire nel pitch dopo un grafico di correlazione? **Feedback:** campione, variabili, periodo, interpretazione limitata e verifica successiva, senza attribuzione causale.

Note di verifica e limiti

Numeri, date e formato dell'esame emersi in R 0–13′ sono incerti e richiedono Canvas ufficiale. La diarizzazione non identifica i docenti o gli studenti. I numeri e le spiegazioni storiche incerte dell'ASR su aziende o modelli non sono trattati come fatti. Le slide 36–48 non sono attribuite a una spiegazione orale. Il testo delle due trascrizioni e delle 55 slide è stato esaminato; gli audio originali non sono stati ascoltati integralmente in questa lavorazione (**direct_audio_review: pending**).

Fonti

1. Archivio: 2026-10-06__spiegazione-estesa.md — testo originale conservato nella stessa cartella